МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск с возвратом

Вариант: 2р

Студент гр. 4343 _____ Чулков В.М.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Вариант

Вар. 2р. Рекурсивный бэктрекинг. Исследование времени выполнения отразмера квадрата.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 11 до N−1$N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера NN. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×77×7 может быть построена из 9 обрезков.

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные:

Размер столешницы - одно целое число N N (2≤N≤202≤N≤20).

Выходные данные:

Одно число KK, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера NN. Далее должны идти KK строк, каждая из которых должна содержать три целых числа x,yx,y и ww, задающие координаты левого верхнего угла (1≤x,y≤N1≤x,y≤N) и длину стороны соответствующего обрезка(квадрата).

Описание работы алгоритма

Для решения задачи реализован рекурсивный алгоритм бэктрекинга. Основная идея метода состоит в том, что решение строится постепенно: на каждом шаге выбирается очередной квадрат, который можно поставить на текущее свободное место, после чего алгоритм рекурсивно достраивает оставшуюся часть большого квадрата. Если текущее состояние приводит к неудачному или заведомо неоптимальному результату, выполняется откат к предыдущему состоянию, и пробуется другой вариант.

Перед запуском основного алгоритма бэктрекинга используется ряд оптимизаций:

1. Для четного N оптимальное решение заведомо известно, поэтому его нахождение выполняется без использования бэктрекинга. Такая оптимизация позволяет найти оптимальное решение линейным способом.

2. Если N составное число, то выполняется деление на его наибольший делитель не равный N и ищется оптимальное разбиение для квадрата полученного размера. Оптимальное решение для такого квадрата будет шаблоном оптимального решения для изначального квадрата. Чтобы найти оптимальное разбиение изначального квадрата достаточно умножить координаты и размеры квадратов разбиения меньшего квадрата на наибольший делитель не равный N изначального квадрата.

3. Для N делящегося на 3 оптимальное решение заведомо известно, поэтому его нахождение также выполняется без использования бэктрекинга. Такая оптимизация позволяет найти оптимальное решение линейным способом. (Это оптимизация является частным случаем оптимизации пункта 2, но в своей реализации не использует бэктрекинг, что упрощает ее работу)

4. Если N простое, то перед выполнением алгоритма бэктрекинга задаются первые 3 наибольших квадрата разбиения, т.к. они также заведомо известны.

5. Если разбиение текущего шага бэктрекинга содержит больше квадратов, чем наилучшее найденное ранее полное разбиение, то эта ветвь завершается. (Основная логика реализованного бэктрекинга)

Способ хранения частичных решений

Для хранения текущего состояния доски используется двумерный список board размера N * N.

Каждая клетка принимает одно из двух значений:

- 0, если клетка свободна;
- 1, если клетка уже занята некоторым квадратом.

Текущее частичное решение хранится в списке cur, который передаётся в рекурсивную функцию. Каждый элемент этого списка представляет собой тройку: [x, y, size] где:

- x - координата левого верхнего угла квадрата по горизонтали;
- y - координата левого верхнего угла квадрата по вертикали;
- size - длина стороны квадрата.

Лучшее найденное решение хранится в списке best_ans.

Текущее минимальное количество квадратов хранится в переменной best[0].

Таким образом, частичное решение хранится явно: в любой момент времени список cur содержит все квадраты, уже поставленные в текущей ветви рекурсии.

Описание рекурсивной функции

Сигнатура: backtracking(cur)

Назначение: Функция выполняет полный перебор возможных способов достроить текущее частичное разбиение квадрата до полного покрытия. Она реализует поиск в глубину с возвратом.

Аргументы: cur - список квадратов, которые уже размещены на доске в текущей ветви поиска.

Возвращаемое значение: Функция ничего не возвращает явно. Все результаты работы сохраняются во внешних переменных:

- best[0] - минимальное количество квадратов в найденном решении;
- best_ans - список квадратов, соответствующий лучшему найденному разбиению.

Принцип работы:

Сначала функция проверяет условие отсечения: если текущее количество квадратов уже не лучше лучшего найденного ответа, дальнейший поиск прекращается.

Затем находится первая свободная клетка. Если свободных клеток не осталось, это означает, что найдено полное покрытие квадрата. В этом случае текущее решение копируется в best_ans, если оно лучше предыдущего.

Если свободная клетка найдена, вычисляется максимальный размер квадрата, который можно поставить в эту позицию. Затем функция перебирает все допустимые размеры от максимального к минимальному. Для каждого размера выполняются следующие действия:

1. Квадрат ставится на поле;
2. Квадрат добавляется в текущее решение cur;
3. Выполняется рекурсивный вызов backtracking(cur);
4. После возврата квадрат удаляется из списка cur;
5. Клетки поля освобождаются.

Так реализуется классическая схема бэктрекинга: выбирается определенное действие, если по ходу рекурсии заведомо виден неоптимальный исход, то функция откатывается назад и переходит к следующему варианту.

Описание вспомогательных функций

Функция is_placeable_at(x, y, size) проверяет, можно ли поставить квадрат размера size в точку (x, y). Она контролирует выход за границы и наличие уже занятых клеток.

Функция place(x, y, size, val) либо размещает квадрат на поле, либо удаляет его. Если val = 1, клетки помечаются как занятые. Если val = 0, клетки снова становятся свободными.

Функция find_first_empty() находит первую свободную клетку на поле. Если таких клеток нет, возвращается [-1, -1].

Функция get_max_square_size(x, y) определяет максимальный размер квадрата, который можно поставить в данную позицию.

Функция check_is_non_prime(N) ищет делитель числа N. Если делитель найден, функция возвращает его, иначе возвращает False.

Функция main(N) управляет всей логикой программы: выбирает нужную ветвь решения в зависимости от чётности и делимости числа N.

Исследование

Для исследования производительности выполнялась серия запусков программы для значений N от 2 до 28. Для каждого значения измерялось время выполнения функции main(N). Затем строился график зависимости времени выполнения от размера квадрата представленный на рисунке 1.

Рисунок 1 - Зависимость времени поиска разбиения от размера квадрата

По полученному графику можно сделать следующие выводы.

Для чётных значений N время выполнения мало, так как решение строится сразу по готовой формуле.

Для значений N, кратных 3, также наблюдается небольшое время выполнения, поскольку используется специальная конструктивная схема.

Для составных чисел, имеющих делитель, время работы снижается благодаря уменьшению задачи и последующему масштабированию результата.

Наибольшее время выполнения наблюдается для простых чисел, поскольку в этом случае используется полный рекурсивный перебор с возвратом. С ростом N число конфигураций увеличивается, и время работы алгоритма возрастает.

Таким образом, эксперимент показывает, что производительность алгоритма зависит не только от размера квадрата, но и от арифметической структуры числа N. Наличие делителей позволяет существенно ускорить поиск решения.

Оценка сложности алгоритма

Временная сложность:

С ростом размера квадрата количество вариантов резко увеличивается, поэтому алгоритм может работать значительно дольше. При этом на практике время уменьшается за счёт оптимизаций: отдельной обработки чётных N, специального случая для чисел, кратных 3, уменьшения задачи для составных чисел, отсечения невыгодных ветвей.

Но скорость работы алгоритма для простых чисел слишком велика, т.к. нет оптимизации для его работы, а в общем случае алгоритм работает долго, потому что он перебирает разные варианты разбиения квадрата. Для каждой свободной позиции он пробует поставить квадрат разного размера, а затем рекурсивно продолжает поиск. Из-за этого в худшем случае время работы растёт очень быстро с увеличением N. Поэтому временную сложность можно оценить как экспоненциальную.

Итог: временная сложность - O(c^n).

Пространственная сложность:

Основная память расходуется на хранение поля board размера N * N, где отмечаются занятые и свободные клетки.

Также память используется для: текущего частичного решения, лучшего найденного решения, стека рекурсии.

Так как основную часть памяти занимает именно поле, итоговая оценка по памяти: $O(N^2)$.

Вывод

В ходе лабораторной работы был реализован алгоритм рекурсивного бэктрекинга для задачи разбиения квадрата на минимальное количество квадратов меньшего размера.

В программе были использованы:

1. частные решения для чётных N.
2. специальная схема для чисел, кратных 3.
3. уменьшение задачи для составных чисел.
4. рекурсивный поиск с возвратом для остальных случаев.

Были рассмотрены способы хранения частичных решений, перечислены использованные оптимизации и дана оценка сложности алгоритма по времени и памяти.

Проведённое исследование показало, что время выполнения резко зависит от структуры числа N: для простых чисел алгоритм работает существенно дольше, чем для чётных и составных чисел, для которых можно применить специальные приёмы ускорения.

Приложение А

Код программы

Код программы main.py:

```
import math

def solve_for_even(n):
    print(4)
    print(1,1,n//2)
    print(1+n//2,1,n//2)
    print(1,1+n//2,n//2)
    print(1+n//2,1+n//2,n//2)
    return

def solve_for_divisable_on_three(N,a):
    b = N//a
    print(a*2)
    print(1,1,b*(a-1))
    for i in range(a):
        print(1+b*(i),1+b*(a-1),b)
        if i != a-1:
            print(1+b*(a-1),1+b*(i),b)

def solve_for_others(N,s):
    board = [[0]*N for _ in range(N)]
    best = [float("+inf")]
    best_ans = []

    def is_placeable_at(x,y,size):
        if size + x > N or size + y > N:
            return False
        for i in range(x,x+size):
            for k in range(y,y+size):
                if board[k][i]:
                    return False
        return True

    def place(x,y,size,val):
        for i in range(x,x+size):
            for k in range(y,y+size):
               board[k][i] = val

    def find_first_empty():
        for i in range(N):
            for k in range(N):
                if not board[k][i]:
                    return [i,k]
        return [-1,-1]

    def get_max_square_size(x,y):
        size = 0
        while size+x <= N and size+y <= N:
```

```
            if is_placeable_at(x,y, size):
                size+=1
            else:
                break
        return size-1


    def backtracking(cur):
        if len(cur)>=best[0]:
            return

        pos = find_first_empty()
        x,y = pos
        if x == -1:
            best[0] = len(cur)
            best_ans.clear()
            best_ans.extend(cur[:])
            return

        max_possible = get_max_square_size(x, y)

        for size in range(max_possible, 0, -1):
            if size == N:
                continue
            if is_placeable_at(x,y,size):
                place(x,y, size, 1)
                cur.append([x,y, size])
                backtracking(cur)
                cur.pop()
                place(x,y, size, 0)

    p = N//2

    start = [[0,0,p+1], [0,p+1,p], [p+1,0,p]]

    for x,y,size in start:
        place(x,y,size,1)

    backtracking(start)

    s.extend(best_ans)


def check_is_non_prime(N):
    for i in range(2,int(math.sqrt(N))+1):
        if N%i==0:
            return i
    return False


def main(N):

    a = check_is_non_prime(N)
```

```
    if N%2==0:
        solve_for_even(N)
    elif a==3:
        solve_for_divisable_on_three(N,a)
    else:
        s = []
        if a:
            reduced = a
            k = N // reduced
            solve_for_others(reduced, s)
            print(len(s))
            for i in s:
                print(i[0]*k, i[1]*k, i[2]*k)
        else:
            solve_for_others(N, s)
            print(len(s))
            for i in s:
                print(*i)



if __name__ == "__main__":
    N = int(input())
    main(N)
```

Код программы graphs.py:

```
import matplotlib.pyplot as plt
from main import main
import time

if __name__ == "__main__":
    metrics = []
    sizes = list(range(2, 28))
    for i in sizes:
        start = time.time()
        print("")
        print(f"---- Solve for N = {i} ----")
        main(i)
        end = time.time()
        metrics.append(end-start)

    plt.figure(figsize=(12, 6))

    plt.plot(sizes, metrics, label="Время поиска", color="black")
    plt.plot(sizes, metrics, "bo", color="red")

    plt.title("Зависимость времени поиска разбиения от размера
квадрата")
    plt.xlabel("Размер квадрата N")
    plt.ylabel("Время выполнения (сек)")
```

```
plt.xticks(sizes)
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()


for x, y in zip(sizes, metrics):
    plt.text(x, y, f"{y:.2f}", fontsize=8)

plt.tight_layout()
plt.show()
```

Приложение Б

Тестирование

1. Минимальный допустимый размер

Вход: N = 2

Ожидаемое поведение: изначальный квадрат разбивается на 4 квадрата размера 1 на 1.

Это минимальный случай, который проверяет корректность работы программы на нижней границе диапазона.

2. Первый нечётный простой размер

Вход: N = 3

Ожидаемое поведение: при помощи оптимизации для квадратов кратности 3 будет мгновенно найдено оптимальное разбиение

Этот случай важен, т. к. демонстрирует корректную работу оптимизации для квадратов кратности меньшей 3.

3. Первый чётный размер, отличный от минимального

Вход: N = 4

Ожидаемое поведение: мгновенное построение ответа из 4 квадратов размера 2 на 2.

Этот тест подтверждает корректность разбиения четных квадратов.

4. Небольшое простое число

Вход: N = 5

Ожидаемое поведение: Использование бэктрекинга без каких-либо оптимизаций по делителям числа.

Проверяется работа рекурсивного поиска для простого числа без упрощения задачи делителем.

5. Число, кратное 3

Вход: N = 9

Ожидаемое поведение: мгновенное нахождение разбиения при помощи оптимизации для квадратов кратных 3.

Важный случай, т. к. проверяется корректность работы оптимизации для квадратов кратных 3

6. Составное число, не равное 2k и не кратное 3

Вход: N = 25

Ожидаемое поведение: используется оптимизация для составных чисел, с выделением наибольшего делителя и рассмотрением квадрат размерности N/a, где a — наибольший делитель N.

Проверяется ветвь уменьшения задачи к квадрату меньшего размера с последующим масштабированием результата.

7. Более крупное простое число

Вход: N = 13 или N = 17

Ожидаемое поведение: Использование бэктрекинга без каких-либо оптимизаций по делителям числа.

Такие тесты проверяют поведение рекурсивного поиска на входах, где пространство вариантов уже заметно возрастает.